

What is JavaScript & V8 Engine Architecture

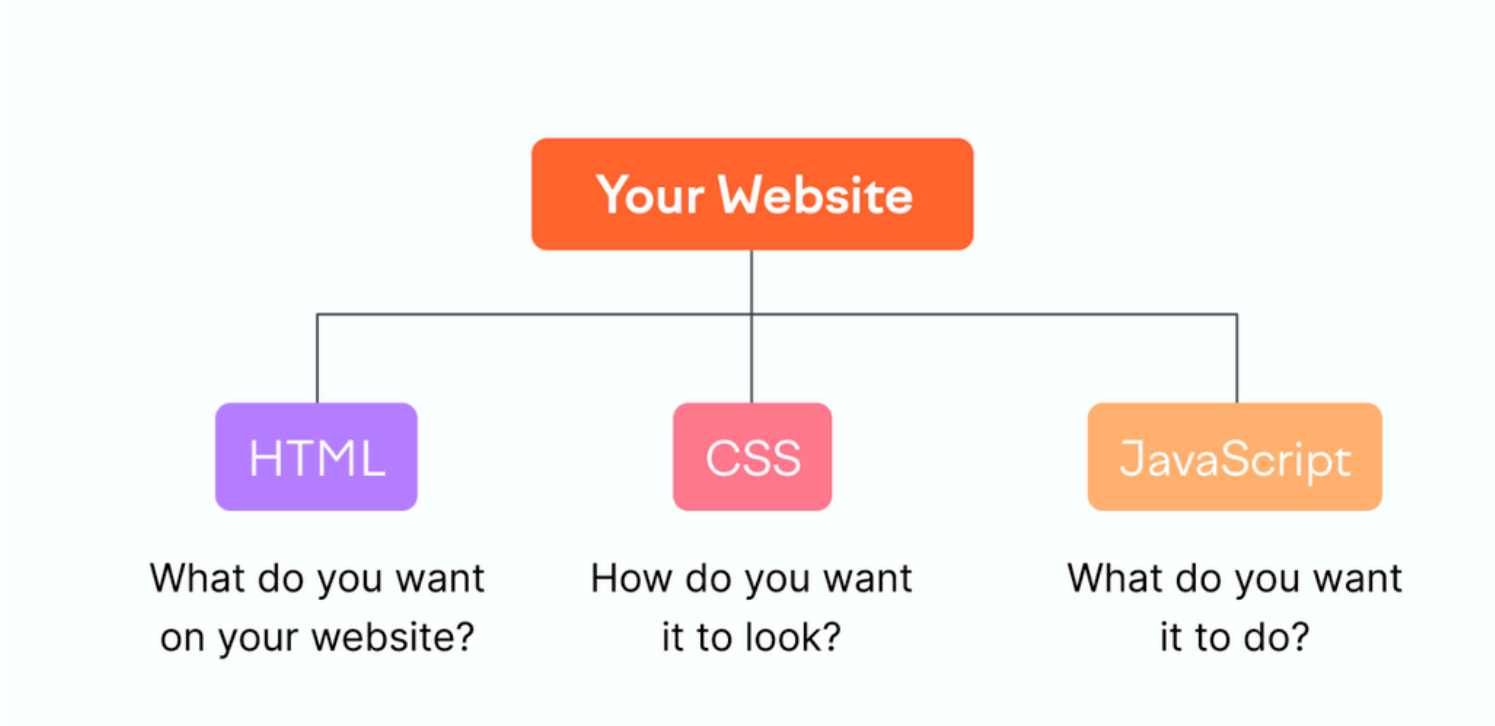
Important Download Links

NodeJS Download URL:	https://nodejs.org/en/download
VSC Download URL:	https://code.visualstudio.com/download

Javascript Notes:

JavaScript is a high-level, interpreted programming language that was originally created to make web pages interactive.

It's one of the core technologies of web development alongside HTML and CSS, but has since expanded far beyond just web browsers.



Features of JavaScript	Features
------------------------	----------

Key Features of JavaScript	• Interpreted Language: JavaScript code is executed directly by the JavaScript engine without needing to be compiled first, making development faster and more flexible. • Dynamic Typing: Variables don't need explicit type declarations. The type is determined at runtime based on the value assigned. • Object-Oriented Programming: Supports objects, classes, inheritance, and encapsulation. You can create objects using constructors, classes, or object literals. • Functional Programming: Functions are first-class citizens, meaning they can be assigned to variables, passed as arguments, and returned from other functions. Supports higher-order functions, and arrow functions. • Asynchronous Programming: Handles operations that take time (like API calls) without blocking the main thread using callbacks, promises, and async/await syntax. • Prototype-Based Inheritance: Objects can inherit directly from other objects through prototypes, providing a flexible inheritance model. • Automatic Memory Management: Features garbage collection to automatically manage memory allocation and deallocation. • Cross-Platform: Runs in browsers, on servers (Node.js), mobile apps, desktop applications, and even IoT devices. • Weak Typing: Automatically converts between different data types when needed, though this can sometimes lead to unexpected behavior. • Client-Side and Server-Side: Originally for browsers, but now used on servers, making it possible to use the same language for full-stack development.
----------------------------	---

JavaScript Engines by Browser

JavaScript Engines by Browser

A comprehensive overview of JS engines powering modern web browsers

BROWSER	JAVASCRIPT ENGINE	DEVELOPER	NOTES
Google Chrome	V8 	Google	High-performance engine with JIT compilation. Also used in Node.js and Electron.
Mozilla Firefox	SpiderMonkey 	Mozilla	First JavaScript engine ever created. Features advanced JIT compilation and WebAssembly support.
Apple Safari	JavaScriptCore (Nitro)	Apple	Optimized for Apple devices.
Microsoft Edge	V8 	Google/Microsoft	Switched from Chakra to V8 in 2020 when adopting Chromium base.
Internet Explorer	Chakra 	Microsoft	Legacy browser, discontinued. Chakra was also used in early Edge versions.
Opera	V8 	Google	Switched from Presto engine to V8/Blink in 2013.
Samsung Internet	V8 	Google	Based on Chromium, popular on Samsung Android devices.
Brave	V8 	Google	Privacy-focused browser built on Chromium.
Vivaldi	V8 	Google	Feature-rich browser based on Chromium.
Arc	V8 	Google	Modern browser with unique UI, built on Chromium.

What is V8 engine?

The **V8 engine** is a high-performance **JavaScript engine** developed by **Google**. It is most famously used in:

- **Google Chrome** (web browser)
- **Node.js** (server-side JavaScript runtime)

What does the V8 engine do?

V8 **executes JavaScript code**. Its main job is to take human-readable JS code and convert it into **machine code** (low-level code understood by the CPU) for **fast execution**.

V8 Engine Internals (Simplified Workflow)

1. **Parser**: Parses JS code → creates an **Abstract Syntax Tree (AST)**
2. **Ignition (Interpreter)**: Converts AST → **Bytecode**
3. **TurboFan (Compiler/Optimizer)**:
 - Detects frequently-used ("hot") code

- Compiles that part of bytecode → **Optimized machine code**

This combination gives a good **balance of speed and efficiency**.

Online Tools

- **AST Explorer:** <https://astexplorer.net/> - Visualize AST in real-time
- **JS AST Viewer:** Interactive AST visualization

See the bytecode

```
node --print-bytecode your-script.js
```

How V8 runs the javascript code:

JS Code (.js)

↓

Parser → AST → Ignition (Interpreter) → Bytecode → Execution -> output

↓

Hot code? → TurboFan (JIT compiler) → Optimized execution: machine code

What the Parser Does?

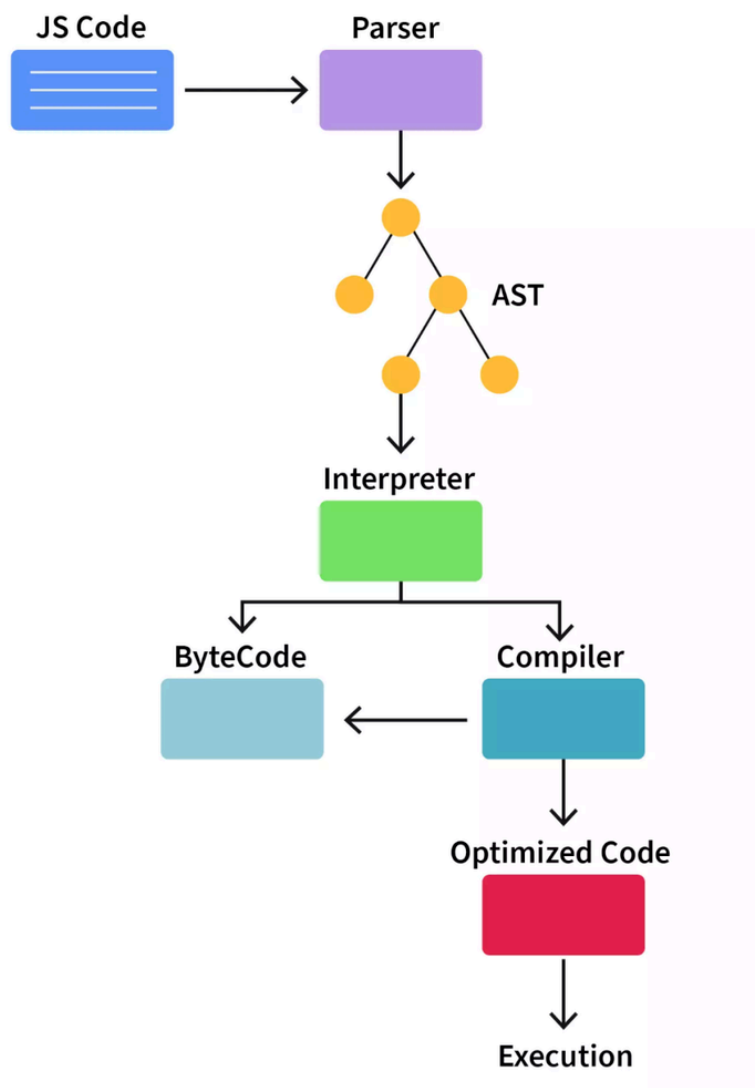
- The Parser is a part of the JavaScript engine (like V8) that:
- Reads raw JavaScript code (your .js file)
- Breaks it into tokens (via the lexer Tokenizer)
- Analyzes the grammar, structure & syntax errors
- Builds an Abstract Syntax Tree (AST) to represent the code

What is a Lexer (aka Tokenizer)?

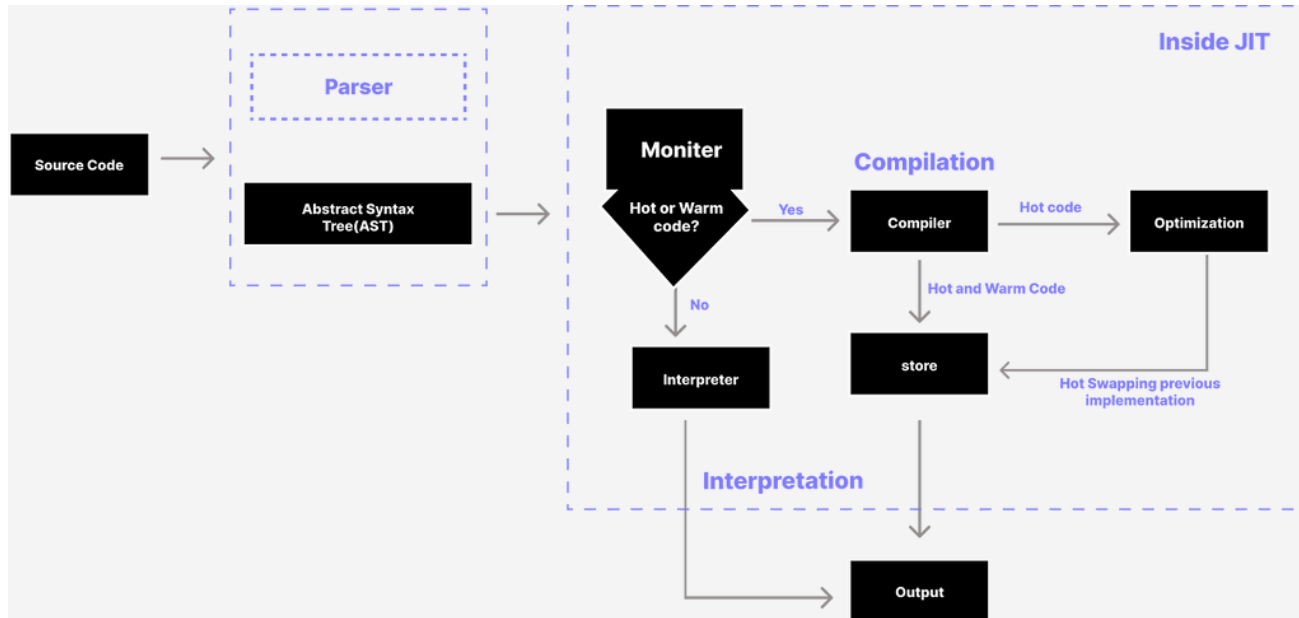
- A Lexer is a component of the JavaScript engine that reads your raw source code and breaks it down into small, meaningful pieces called tokens.
- Purpose of the Lexer
- Converts the code from a stream of characters → to a stream of tokens
- Removes whitespaces and comments (ignored at this stage)
- Makes it easier for the Parser to understand the structure of your code

AST: Abstract Syntax Tree (this is also called output of the Parser)

<https://astexplorer.net/>



Javascript code lifecycle/workflow of Javascript code:

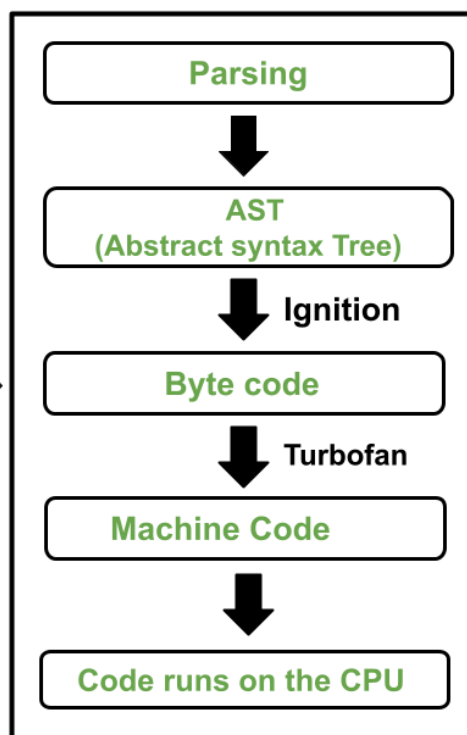


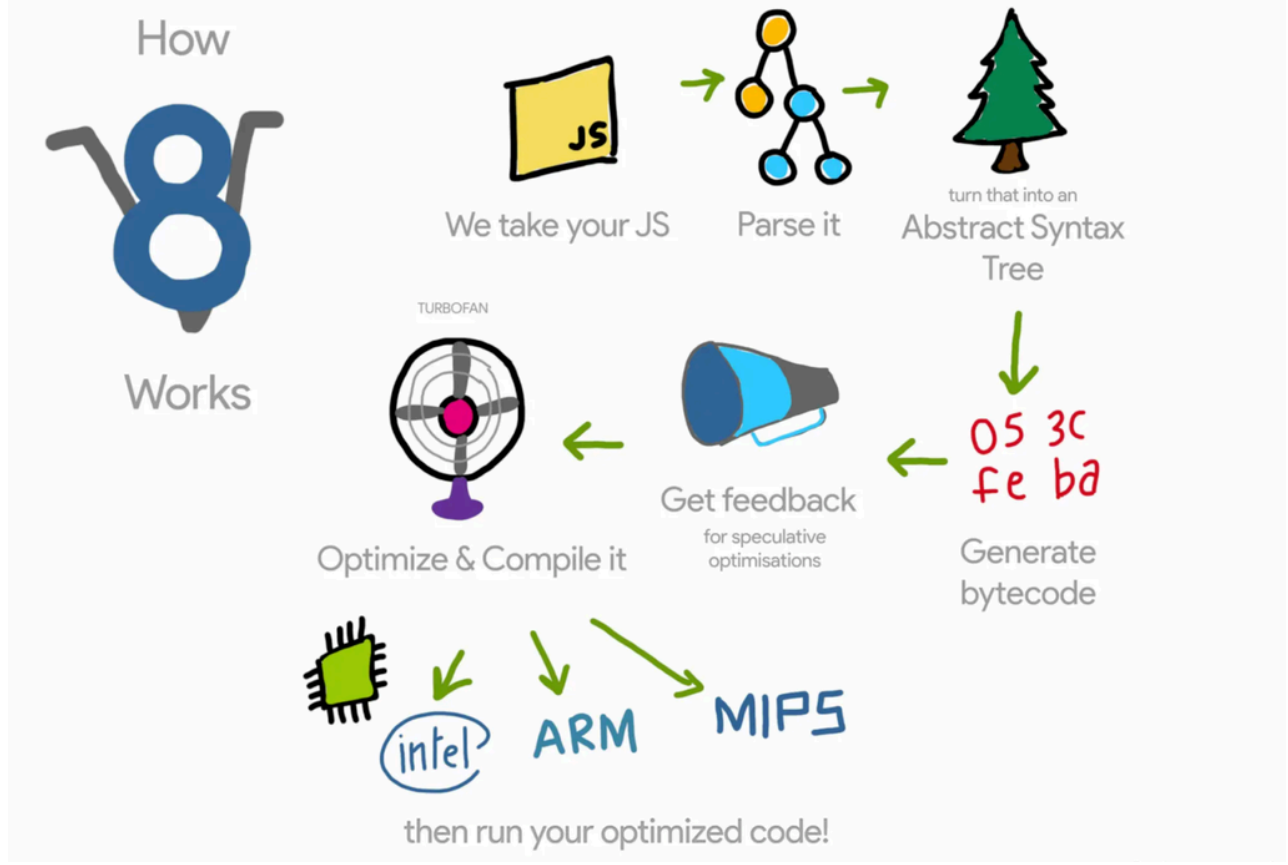
Node.js environment



JS Source code

Chrome V8 engine





How to See V8 Internals (including JIT output):

How to See V8 Internals (including JIT output)		
1. Run Node.js with V8 Debug Flags	<code>node --trace-opt --trace-deopt --trace-ic yourfile.js</code>	<code>--trace-opt</code> When a function gets optimized by TurboFan <code>--trace-deopt</code> When optimization is cancelled (de-optimized) <code>--trace-ic</code> Inline caching behavior
2. See Bytecode (Ignition)	<code>node --print-bytecode yourfile.js</code>	This prints the low-level bytecode generated by Ignition for each function.
Example: What You Might See function add(a, b) { return a + b; } for (let i = 0; i < 1000000; i++) { add(2, 3); }	<code>node --trace-opt add.js</code>	---That means the add function was marked "hot" and optimized by TurboFan (JIT compiler).

